

04 — Database Migrations

Table of Contents

1. [Learning Objectives](#)
2. [Why Migrations Matter](#)
3. [Migration Tools Overview](#)
4. [Zero-Downtime Migration Patterns](#)
5. [Adding a Column \(Safe\)](#)
6. [Adding a NOT NULL Column \(Dangerous → Safe\)](#)
7. [Adding an Index CONCURRENTLY](#)
8. [Renaming a Column \(3-Step\)](#)
9. [Backfilling Large Tables Without Locks](#)
10. [Blue-Green Schema Migration](#)
11. [Common Mistakes](#)
12. [Best Practices](#)
13. [Performance Considerations](#)
14. [Interview Questions & Answers](#)
15. [Exercises with Solutions](#)
16. [Cross-References](#)

Learning Objectives

By the end of this section you will be able to:

- Choose the right migration tool for your stack
- Explain which DDL operations acquire table-level locks in PostgreSQL
- Apply the safe pattern for adding a NOT NULL column to a live table
- Create indexes concurrently without blocking reads or writes
- Rename a column in three backward-compatible steps
- Backfill millions of rows without causing lock contention
- Describe the blue-green schema migration strategy

Why Migrations Matter

Every application change that modifies the database schema is a migration. Done wrong, a migration can:

- Lock a table for minutes, causing a full application outage
- Break backward compatibility (old application code reading new schema, or vice versa during deploy)
- Silently corrupt data (changing a column type with implicit cast)
- Leave the database in a partially migrated state if interrupted

PostgreSQL is ACID-compliant, which means DDL statements (like `ALTER TABLE`) run inside transactions. However, **some operations acquire aggressive locks** that block all reads and writes.

PostgreSQL Lock Levels for Common DDL

Operation	Lock Level	Blocks
<code>ALTER TABLE ADD COLUMN</code> (no default, no NOT NULL)	ShareRowExclusive	DML

ALTER TABLE ADD COLUMN ... DEFAULT (Pg < 11)	AccessExclusive	ALL
ALTER TABLE ADD COLUMN ... DEFAULT (Pg >= 11)	ShareRowExclusive	DML only
ALTER TABLE SET NOT NULL	ShareRowExclusive	DML
ALTER TABLE ADD CONSTRAINT CHECK	ShareRowExclusive	DML
ALTER TABLE ADD CONSTRAINT CHECK NOT VALID	ShareRowExclusive brief	Minimal
ALTER TABLE VALIDATE CONSTRAINT	ShareUpdateExclusive	Only other schema changes
CREATE INDEX	ShareLock	All writes
CREATE INDEX CONCURRENTLY	None (phases)	Almost nothing
DROP INDEX	AccessExclusive	ALL
DROP INDEX CONCURRENTLY	ShareUpdateExclusive	Only other schema changes
ALTER TABLE RENAME COLUMN	AccessExclusive	ALL
TRUNCATE	AccessExclusive	ALL

Migration Tools Overview

Flyway

- Java-based, SQL-first migration tool
- Migrations are numbered SQL files: `V1__Create_orders.sql` , `V2__Add_index.sql`
- Supports Java-based migrations for complex data operations
- Excellent Spring Boot integration

```
# application.properties
spring.flyway.locations=classpath:db/migration
spring.flyway.baseline-on-migrate=true
spring.flyway.validate-on-migrate=true
```

```
-- db/migration/V3__Add_tenant_id_to_orders.sql
-- Safe zero-downtime migration
ALTER TABLE orders ADD COLUMN tenant_id BIGINT;
CREATE INDEX CONCURRENTLY idx_orders_tenant_id ON orders(tenant_id);
```

Liquibase

- Java-based, supports XML/YAML/JSON/SQL formats
- Powerful changeset system with rollback support
- Better for teams needing audit trails

```
# db/changelog/003_add_tenant_id.yaml
databaseChangeLog:
  - changeSet:
      id: 003_add_tenant_id
      author: dev_team
      changes:
        - addColumn:
            tableName: orders
            columns:
              - column:
                  name: tenant_id
                  type: BIGINT
        - createIndex:
            indexName: idx_orders_tenant_id
            tableName: orders
            columns:
              - column:
                  name: tenant_id
rollback:
  - dropIndex:
      indexName: idx_orders_tenant_id
      tableName: orders
  - dropColumn:
      tableName: orders
      columnName: tenant_id
```

Alembic (Python / SQLAlchemy)

- Python migration tool, tightly integrated with SQLAlchemy
- Auto-generates migrations from model diffs

```
alembic init alembic
alembic revision --autogenerate -m "add tenant_id to orders"
alembic upgrade head
alembic downgrade -1
alembic history
alembic current
```

```
# alembic/versions/003_add_tenant_id.py
from alembic import op
import sqlalchemy as sa

revision = '003_add_tenant_id'
down_revision = '002_add_index'

def upgrade():
    # Phase 1: Add nullable column (safe, no lock)
    op.add_column('orders', sa.Column('tenant_id', sa.BigInteger(), nullable=True))

    # Phase 2: Create index concurrently (cannot run inside transaction)
```

```

    op.execute("COMMIT") # close Alembic's implicit transaction
    op.execute("CREATE INDEX CONCURRENTLY idx_orders_tenant_id ON
orders(tenant_id)")

def downgrade():
    op.execute("COMMIT")
    op.execute("DROP INDEX CONCURRENTLY IF EXISTS idx_orders_tenant_id")
    op.drop_column('orders', 'tenant_id')

```

golang-migrate

- CLI and library for Go applications
- Source drivers: file, GitHub, S3, GCS
- Database drivers: PostgreSQL, MySQL, SQLite, etc.

```

# Install
go install -tags 'postgres' github.com/golang-migrate/migrate/v4/cmd/migrate@latest

# Create migration
migrate create -ext sql -dir db/migrations -seq add_tenant_id

# Run
migrate -path db/migrations -database "postgresql://user:pass@host/db?
sslmode=disable" up
migrate -path db/migrations -database "..." down 1
migrate -path db/migrations -database "..." version

```

```

-- db/migrations/000003_add_tenant_id.up.sql
ALTER TABLE orders ADD COLUMN tenant_id BIGINT;

-- db/migrations/000003_add_tenant_id.down.sql
ALTER TABLE orders DROP COLUMN IF EXISTS tenant_id;

```

```

// Embedded in Go application
import (
    "github.com/golang-migrate/migrate/v4"
    _ "github.com/golang-migrate/migrate/v4/database/postgres"
    _ "github.com/golang-migrate/migrate/v4/source/file"
)

func runMigrations(databaseURL string) error {
    m, err := migrate.New("file://db/migrations", databaseURL)
    if err != nil {
        return fmt.Errorf("creating migrator: %w", err)
    }
    defer m.Close()

    if err := m.Up(); err != nil && err != migrate.ErrNoChange {
        return fmt.Errorf("running migrations: %w", err)
    }
}

```

```
}  
  return nil  
}
```

Zero-Downtime Migration Patterns

The core principle: **deploy application code that works with both the old and new schema, then migrate the schema, then remove the backward-compatibility shim.**

```
Phase 1: Deploy v2 app (reads new column if present, falls back to old)  
Phase 2: Run schema migration  
Phase 3: Deploy v3 app (assumes new schema, removes fallback code)
```

Adding a Column (Safe)

PostgreSQL >= 11 with a Volatile Default

```
-- PostgreSQL 11+: Adding a column with a constant/stable default  
-- does NOT rewrite the table – metadata-only change  
ALTER TABLE orders ADD COLUMN discount_pct NUMERIC(5,2) DEFAULT 0.00;  
-- Instant, no table rewrite, ShareRowExclusive lock (brief)  
  
-- PostgreSQL 10 and below: ANY default causes a full table rewrite  
-- (AccessExclusive lock for minutes on large tables)
```

Safe Pattern for Any PostgreSQL Version

```
-- Step 1: Add nullable column with no default (instant)  
ALTER TABLE orders ADD COLUMN discount_pct NUMERIC(5,2);  
  
-- Step 2: Backfill in batches (see section below)  
UPDATE orders SET discount_pct = 0.00  
WHERE id BETWEEN 1 AND 100000 AND discount_pct IS NULL;  
-- ... repeat in batches  
  
-- Step 3: Set DEFAULT (metadata only, instant)  
ALTER TABLE orders ALTER COLUMN discount_pct SET DEFAULT 0.00;
```

Adding a NOT NULL Column (Dangerous → Safe)

Why It Is Dangerous

```
-- DANGEROUS on large tables – full table scan + AccessExclusive lock  
ALTER TABLE orders ADD COLUMN status VARCHAR(20) NOT NULL DEFAULT 'pending';  
-- PostgreSQL < 11: rewrites entire table, holds lock for minutes
```

Safe Approach (5-Phase Pattern)

```
-- Phase 1: Add column as NULLABLE (instant)
ALTER TABLE orders ADD COLUMN status VARCHAR(20);
-- Takes: ShareRowExclusive lock for milliseconds

-- Phase 2: Set default value for new rows (instant, metadata)
ALTER TABLE orders ALTER COLUMN status SET DEFAULT 'pending';
-- Takes: ShareRowExclusive lock for milliseconds

-- Phase 3: Backfill existing rows in batches (no table lock)
DO $$
DECLARE
    batch_size INT := 10000;
    last_id    BIGINT := 0;
    max_id     BIGINT;
BEGIN
    SELECT MAX(id) INTO max_id FROM orders;
    WHILE last_id < max_id LOOP
        UPDATE orders
        SET status = 'pending'
        WHERE id > last_id AND id <= last_id + batch_size AND status IS NULL;
        last_id := last_id + batch_size;
        PERFORM pg_sleep(0.01); -- brief pause to reduce lock contention
    END LOOP;
END $$;

-- Phase 4: Add NOT VALID constraint (acquires brief lock, skips scan)
ALTER TABLE orders ADD CONSTRAINT orders_status_not_null
    CHECK (status IS NOT NULL) NOT VALID;
-- NOT VALID: constraint is enforced for new rows only, skips existing rows

-- Phase 5: Validate constraint (ShareUpdateExclusive – does NOT block DML)
ALTER TABLE orders VALIDATE CONSTRAINT orders_status_not_null;
-- This scans the entire table but only holds ShareUpdateExclusive lock
-- which does NOT block reads or writes!

-- Phase 6 (Optional): Convert to real NOT NULL if needed
-- (Pg 12+ can detect that CHECK NOT NULL constraint exists and make this fast)
ALTER TABLE orders ALTER COLUMN status SET NOT NULL;
-- PostgreSQL 12+: this is instant if a validated NOT NULL CHECK exists
```

Adding an Index CONCURRENTLY

The Problem with Regular CREATE INDEX

```
-- BLOCKS ALL WRITES for the entire duration (minutes on large tables)
CREATE INDEX idx_orders_customer ON orders(customer_id);
```

CREATE INDEX CONCURRENTLY

```
-- Does NOT block reads or writes
-- Takes longer (multiple passes), but safe for production
CREATE INDEX CONCURRENTLY idx_orders_customer ON orders(customer_id);

-- Works for partial indexes too
CREATE INDEX CONCURRENTLY idx_orders_pending
ON orders(customer_id, created_at)
WHERE status = 'pending';

-- CANNOT be run inside a transaction block
BEGIN;
CREATE INDEX CONCURRENTLY ...; -- ERROR: CREATE INDEX CONCURRENTLY cannot run
inside a transaction block
COMMIT;
```

Alembic: Running CONCURRENTLY Outside Transaction

```
# In Alembic, you must disable the autobegin transaction:
def upgrade():
    # Tell Alembic not to wrap this in a transaction
    op.execute("COMMIT") # close any open transaction

    # Now run concurrently
    op.execute("""
        CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_orders_customer
        ON orders(customer_id)
    """)

    # Start a new transaction for the rest of the migration
    op.execute("BEGIN")
```

golang-migrate: CONCURRENTLY in Its Own File

```
-- 000010_add_order_customer_index.up.sql
-- golang-migrate wraps each file in a transaction by default.
-- To disable, add this directive:
-- migrate:disable_ddl_transaction

CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_orders_customer ON orders(customer_id);
```

Handling a Failed CONCURRENT Index Build

```
-- A failed concurrent index build leaves an INVALID index
SELECT indexname, indexdef
FROM pg_indexes
```

```

WHERE tablename = 'orders' AND indexname = 'idx_orders_customer';

-- Check for invalid indexes
SELECT schemaname, tablename, indexname
FROM pg_indexes
WHERE tablename = 'orders'
AND NOT pg_catalog.pg_index.indisvalid
FROM pg_catalog.pg_index
JOIN pg_catalog.pg_class ON pg_class.oid = pg_index.indexrelid
WHERE pg_class.relname = 'idx_orders_customer';

-- Drop and rebuild
DROP INDEX CONCURRENTLY IF EXISTS idx_orders_customer;
CREATE INDEX CONCURRENTLY idx_orders_customer ON orders(customer_id);

```

Renaming a Column (3-Step)

Directly renaming a column (`ALTER TABLE RENAME COLUMN`) acquires `AccessExclusive` lock and breaks any deployed application code that references the old column name. The safe approach takes 3 deployment phases.

Step 1 — Add New Column, Dual-Write

```

-- Migration: Add new column
ALTER TABLE orders ADD COLUMN customer_identifier BIGINT;

-- Application code (v2): write to BOTH columns, read from OLD column
-- In your ORM/query layer:
INSERT INTO orders (customer_id, customer_identifier, total)
VALUES ($1, $1, $2) -- write both

UPDATE orders SET customer_identifier = customer_id
WHERE customer_identifier IS NULL; -- backfill existing

```

Step 2 — Read from New Column, Stop Writing to Old

```

-- Backfill complete check
SELECT COUNT(*) FROM orders WHERE customer_identifier IS NULL;
-- Must be 0 before proceeding

-- Application code (v3): write to BOTH, read from NEW column
-- Gradually shift reads to customer_identifier

```

Step 3 — Drop Old Column

```

-- Only after ALL application instances are reading from customer_identifier
ALTER TABLE orders DROP COLUMN customer_id;

```



```
-- Can also do: ALTER TABLE orders RENAME COLUMN customer_identifier TO customer_id
```

Backfilling Large Tables Without Locks

Batch Backfill Script (SQL)

```
DO $$
DECLARE
    batch_size INT := 5000;
    start_id BIGINT := 0;
    end_id BIGINT;
    max_id BIGINT;
    rows_updated INT;
BEGIN
    SELECT COALESCE(MAX(id), 0) INTO max_id FROM orders;
    RAISE NOTICE 'Max ID: %, Batch size: %', max_id, batch_size;

    LOOP
        end_id := start_id + batch_size;

        UPDATE orders
        SET status = COALESCE(status, 'legacy')
        WHERE id > start_id
            AND id <= end_id
            AND status IS NULL;

        GET DIAGNOSTICS rows_updated = ROW_COUNT;
        RAISE NOTICE 'Processed IDs % - %, updated % rows', start_id, end_id,
rows_updated;

        start_id := end_id;
        EXIT WHEN start_id > max_id;

        -- Throttle: sleep 10ms between batches
        PERFORM pg_sleep(0.01);
    END LOOP;
    RAISE NOTICE 'Backfill complete';
END $$;
```

Python Batch Backfill

```
import psycopg2
import time
import logging

logger = logging.getLogger(__name__)

def backfill_status(dsn: str, batch_size: int = 5000, sleep_ms: int = 10):
    """Backfill orders.status in small batches without holding locks."""
```

```

conn = psycopg2.connect(dsn)
conn.autocommit = True # Each batch is its own transaction

with conn.cursor() as cur:
    cur.execute("SELECT MAX(id) FROM orders")
    max_id = cur.fetchone()[0] or 0

start_id = 0
total_updated = 0

while start_id <= max_id:
    end_id = start_id + batch_size

    with psycopg2.connect(dsn) as batch_conn:
        with batch_conn.cursor() as cur:
            cur.execute("""
                UPDATE orders
                SET status = 'legacy'
                WHERE id > %s AND id <= %s AND status IS NULL
            """, (start_id, end_id))
            updated = cur.rowcount
            batch_conn.commit()

    total_updated += updated
    logger.info(f"Backfilled IDs {start_id}-{end_id}: {updated} rows (total: {total_updated})")

    start_id = end_id
    time.sleep(sleep_ms / 1000)

logger.info(f"Backfill complete: {total_updated} total rows updated")

```

Key Principles for Safe Backfills

1. Use small batches (1000–10000 rows) — smaller batches mean shorter lock holds
2. Sleep between batches to allow replication lag to catch up
3. Use `WHERE id > $start AND id <= $end` — range scans on the primary key
4. Run in autocommit mode or commit each batch separately
5. Never use `UPDATE orders SET status = 'x'` without a `WHERE` — this locks the entire table
6. Monitor replication lag during the backfill

Blue-Green Schema Migration

Concept

Run two identical database schemas side by side. Route traffic to the "blue" (current) while preparing "green" (new). Switch traffic when ready.

```

Blue (current):
  orders table: id, customer_id, total, status

```

Green (new):
orders table: id, customer_id, total, status, tenant_id

Dual-write phase: writes go to BOTH blue and green

Switch: reads flip from blue to green

Cleanup: drop blue schema

Implementation with PostgreSQL Schemas

```
-- Create green schema
CREATE SCHEMA green;

-- Create new table structure in green
CREATE TABLE green.orders (LIKE public.orders INCLUDING ALL);
ALTER TABLE green.orders ADD COLUMN tenant_id BIGINT;

-- Create a trigger on public.orders to dual-write to green
CREATE OR REPLACE FUNCTION sync_orders_to_green()
RETURNS TRIGGER AS $$
BEGIN
    IF TG_OP = 'INSERT' THEN
        INSERT INTO green.orders SELECT NEW.*, NULL::BIGINT;
    ELSIF TG_OP = 'UPDATE' THEN
        UPDATE green.orders SET
            customer_id = NEW.customer_id,
            total = NEW.total,
            status = NEW.status
            WHERE id = NEW.id;
    ELSIF TG_OP = 'DELETE' THEN
        DELETE FROM green.orders WHERE id = OLD.id;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER orders_dual_write
AFTER INSERT OR UPDATE OR DELETE ON public.orders
FOR EACH ROW EXECUTE FUNCTION sync_orders_to_green();

-- Backfill historical data
INSERT INTO green.orders
SELECT *, NULL::BIGINT FROM public.orders
WHERE id NOT IN (SELECT id FROM green.orders);

-- Application reads: switch search_path
SET search_path = green, public;

-- After validation: promote green to public
-- (rename schemas – requires brief AccessExclusive lock on schema rename)
```

Common Mistakes

1. Running `CREATE INDEX` (not `CONCURRENTLY`) on a live production table — blocks all writes.
 2. Wrapping `CREATE INDEX CONCURRENTLY` in a transaction — causes an error; must run outside a transaction.
 3. `ALTER TABLE ... ADD COLUMN ... NOT NULL` without a default or backfill on Pg < 11 — full table rewrite.
 4. Running a migration script that fails halfway without rollback — leaves the schema in an inconsistent state. Always wrap migrations in transactions where possible.
 5. Not testing migrations on production-sized data — a migration that takes 1 ms on a 1000-row test DB can take 30 minutes on a 100M-row production table.
 6. Renaming a column in a single step — breaks currently deployed application code.
 7. Forgetting to update sequences/defaults after column changes.
 8. Running `VACUUM FULL` as part of a migration — it acquires `AccessExclusive` lock and rewrites the entire table.
-

Best Practices

- Test all migrations on a production-data-sized clone before deploying
 - Use `CREATE INDEX CONCURRENTLY` for all new indexes on existing tables
 - Split large migrations into multiple PRs/deloys (add column → backfill → add constraint)
 - Version-control all migrations in the repository
 - Never edit a migration file after it has been applied to production
 - Use `IF NOT EXISTS` / `IF EXISTS` in migrations for idempotency
 - Set a migration execution timeout to detect runaway migrations early
 - Keep a rollback plan for every migration (test `migrate down`)
 - Monitor table bloat after backfills — run `VACUUM ANALYZE` afterward
-

Performance Considerations

Lock Wait Timeout in Migrations

```
-- In your migration script, set a lock wait timeout
-- so the migration fails fast instead of waiting indefinitely
SET lock_timeout = '5s';
ALTER TABLE orders ADD COLUMN new_col TEXT;
-- If lock cannot be acquired in 5s, the statement fails
-- Retry during low-traffic window
```

Estimating Backfill Duration

```
-- Sample 1000 rows to estimate per-row cost
EXPLAIN (ANALYZE, BUFFERS)
```

```
UPDATE orders SET status = 'legacy'
WHERE id BETWEEN 1 AND 1000 AND status IS NULL;

-- Example output: actual time=5.2ms for 1000 rows
-- Total rows to backfill: 50,000,000
-- Estimated time: (50,000,000 / 1000) * 5.2ms = 260 seconds = ~4.3 minutes
-- With 10ms sleep per batch: 260 + (50,000,000/1000 * 10ms) = 760 seconds = ~13
minutes
```

Interview Questions & Answers

Q1: Why is `ALTER TABLE ... ADD COLUMN ... NOT NULL DEFAULT 'x'` dangerous on PostgreSQL < 11?

A: On PostgreSQL versions before 11, any `ADD COLUMN` with a non-null default causes a full table rewrite — every row in the table must be physically updated to include the new column value. This acquires an `AccessExclusive` lock that blocks all reads AND writes for the entire duration of the rewrite, which could be minutes for a large table. PostgreSQL 11+ optimized this to a metadata-only change for constant defaults, but volatile functions (e.g., `NOW()` , sequences) still cause rewrites.

Q2: Why can't `CREATE INDEX CONCURRENTLY` run inside a transaction?

A: `CREATE INDEX CONCURRENTLY` works in multiple phases: it first creates an empty index, then scans the table twice (to catch changes that happened during the first scan), and finally makes the index live. Between phases, it must release and reacquire locks. A regular transaction holds its locks from `BEGIN` to `COMMIT` , which would deadlock with the concurrent build phases. PostgreSQL therefore requires it to run outside any transaction block.

Q3: What is the 3-step rename column pattern and why is it necessary?

A: `ALTER TABLE RENAME COLUMN` acquires `AccessExclusive` lock (blocks all reads and writes) and immediately breaks deployed application code that references the old column name. The 3-step pattern is: (1) Add the new column, dual-write to both columns, backfill. (2) Switch reads to the new column, continue dual-writing. (3) Drop the old column after all application instances use the new name. This allows zero-downtime deployment with backward compatibility at each step.

Q4: What is `NOT VALID` in a constraint and why is it useful for migrations?

A: `ALTER TABLE ... ADD CONSTRAINT ... NOT VALID` creates the constraint but does not scan existing rows — it only enforces the constraint on new inserts and updates. This acquires a brief `ShareRowExclusive` lock instead of scanning the whole table. A subsequent `ALTER TABLE VALIDATE CONSTRAINT` then scans the table but holds only `ShareUpdateExclusive` lock, which does not block reads or writes. This splits one potentially-blocking operation into two non-blocking ones.

Q5: What batch size is appropriate for a backfill operation?

A: Typically 1,000–10,000 rows per batch, depending on row size and server load. Smaller batches mean shorter lock holds per batch transaction, allowing other transactions to interleave. Add a small sleep (10–50ms) between batches to reduce I/O pressure and give replication time to catch up. Monitor replication lag during the backfill and increase sleep if lag grows.

Q6: How does Flyway differ from Alembic?

A: Flyway is language-agnostic (SQL-first) and version-number-based (`V1__` , `V2__`), making it simple but less flexible. Alembic is Python/SQLAlchemy-integrated and supports auto-generation of migrations from model diffs. Alembic migrations are written as Python functions with `upgrade()` and `downgrade()` methods, allowing arbitrary Python logic (useful for complex data migrations). Flyway is preferred in Java/Kotlin projects; Alembic in Python/SQLAlchemy projects.

Q7: How do you handle a migration that fails halfway through?

A: Most DDL in PostgreSQL is transactional (unlike MySQL), so wrapping the migration in a transaction means PostgreSQL rolls it back automatically if the migration script fails. For operations that cannot be transactional (`CREATE INDEX CONCURRENTLY` , `VACUUM`), run them separately and make them idempotent with `IF NOT EXISTS` / `IF EXISTS` . Track migration state in a migration table (Flyway/Alembic do this) so you know exactly which migration failed and where to resume.

Q8: What monitoring should you do during a large migration?

A: Monitor: (1) `pg_stat_activity` for blocking queries and lock waits. (2) `pg_locks` for lock conflicts. (3) Replication lag (`pg_stat_replication`) — large backfills generate WAL that replicas must replay. (4) Table bloat — `UPDATE` operations create dead tuples requiring later `VACUUM`. (5) CPU and I/O on the database server. Set `lock_timeout = '5s'` in the migration to fail fast rather than blocking indefinitely.

Exercises with Solutions

Exercise 1: Safe NOT NULL Migration

Problem: You need to add a `tenant_id BIGINT NOT NULL` column to the `orders` table (5 million rows) with zero downtime. Write the complete migration plan.

Solution:

```
-- == Phase 1: Add nullable column (instant, ~10ms) ==
ALTER TABLE orders ADD COLUMN tenant_id BIGINT;

-- == Phase 2: Set default for new rows (instant) ==
ALTER TABLE orders ALTER COLUMN tenant_id SET DEFAULT 1; -- default tenant

-- == Phase 3: Backfill in batches (no table lock) ==
DO $$
DECLARE
    batch_size INT := 10000;
    start_id BIGINT := 0;
    max_id BIGINT;
BEGIN
    SELECT MAX(id) INTO max_id FROM orders;
    WHILE start_id < max_id LOOP
        UPDATE orders SET tenant_id = 1
        WHERE id > start_id AND id <= start_id + batch_size AND tenant_id IS NULL;
        start_id := start_id + batch_size;
        PERFORM pg_sleep(0.01);
    END LOOP;
END $$;
```

```

-- Verify backfill complete
SELECT COUNT(*) FROM orders WHERE tenant_id IS NULL; -- Must be 0

-- == Phase 4: Add NOT VALID constraint (instant) ==
ALTER TABLE orders ADD CONSTRAINT orders_tenant_id_nn
    CHECK (tenant_id IS NOT NULL) NOT VALID;

-- == Phase 5: Validate constraint (non-blocking scan) ==
ALTER TABLE orders VALIDATE CONSTRAINT orders_tenant_id_nn;

-- == Phase 6: Promote to real NOT NULL (instant in Pg 12+) ==
ALTER TABLE orders ALTER COLUMN tenant_id SET NOT NULL;
ALTER TABLE orders DROP CONSTRAINT orders_tenant_id_nn;

-- == Phase 7: Add index ==
CREATE INDEX CONCURRENTLY idx_orders_tenant_id ON orders(tenant_id);

```

Exercise 2: Write Alembic Migration for CONCURRENTLY Index

Problem: Write an Alembic migration that adds an index on `orders(status, created_at)` concurrently (must work outside a transaction).

Solution:

```

from alembic import op

revision = '005_add_orders_status_created_idx'
down_revision = '004_add_tenant_id'
branch_labels = None
depends_on = None

def upgrade():
    # Must run outside any transaction for CONCURRENTLY
    connection = op.get_bind()
    connection.execute(sa.text("COMMIT")) # close Alembic's transaction

    op.execute(sa.text("""
        CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_orders_status_created
        ON orders(status, created_at DESC)
        WHERE status IN ('pending', 'processing')
    """))

def downgrade():
    connection = op.get_bind()
    connection.execute(sa.text("COMMIT"))

    op.execute(sa.text("""
        DROP INDEX CONCURRENTLY IF EXISTS idx_orders_status_created
    """))

```

Cross-References

- `01_connection_pooling_deep.md` — `CREATE INDEX CONCURRENTLY` cannot run through PgBouncer in transaction mode (needs session mode or direct connection)
- `05_api_database_patterns.md` — Adding partial indexes for soft deletes and API filters
- `06_multi_tenant_architectures.md` — Adding `tenant_id` column and RLS policies
- `08_scaling_patterns.md` — Partitioning strategy as a schema migration
- `09_java_spring_postgresql.md` — Flyway with Spring Boot auto-migration
- `11_python_postgresql.md` — Alembic in Django and Flask applications
- `12_go_postgresql.md` — golang-migrate integration patterns